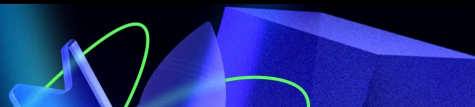


Хабр

Весна новых возможностей

Свежие IT-события уже в Хабре. Каждый день.



Хабр



Чему научиться в этом году?



merra123

13 часов назад

Как я реализовал Blue-Green деплой с нулевым даунтаймом на Docker Compose

🔗 Средний 🕒 5 мин 👁 6.2K

Python*, Django*, Веб-разработка*, Говнокод, Микросервисы*

Кейс

Из песочницы

Недавно я внедрил blue-green деплой в проде. Реализация довольно простая и кастомная, но справляется со своей задачей на ура! Также сообщу, что используется обычный докер композ на виртуалке - возможно, кому-то такой подход будет полезен.

Для фоновых процессов (воркеров)

В приложение добавляется специальный инфраструктурный singleton класс с флагом `is_accepting`, и обертка на consumers. В каждом консьюмере перед обработкой проверяем этот флаг: если `True` - обрабатываем задачу, если `False` - переносим задачу на повторную обработку (например, в rabbitmq делаем сразу `ack(requeue=True)`)

```
class ShutdownGuard:
    def __init__(self) -> None:
        self._accepting = True

    @property
    def is_accepting(self) -> bool:
        return self._accepting

    def start_shutdown(self) -> None:
        self._accepting = False
        logger.info("%s: stopped accepting new tasks", self.__class__.__name__)
```

вот пример такого синглтона на ЯП python.

```
def graceful(guard: ShutdownGuard):
    def decorator(handler):
        @wraps(handler)
        async def wrapper(*args, **kwargs):
            if not guard.is_accepting:
                raise NackMessage(requeue=True)
            return await handler(*args, **kwargs)

        return wrapper

    return decorator
```

Пример обертки. Если `is_accepting=False` - отправляем задачи обратно в очередь.

Когда сервис получает `sigterm` сигнал, этот singleton переключает `is_accepting` в `False`. После переключения добавляем ожидание на время максимального выполнения задачи (+5-10 секунд), и в контейнере обязательно указываем `graceful timeout` на 5-10 секунд больше этого значения.

```
class FastStreamApp:
    def __init__(self):
        log_queue = queue.Queue(maxsize=10000)
        root_logger = setup_queue_logger(log_queue, level=logging.INFO)
        handler = create_console_handler(settings.log_format, logging.INFO)
        self.queue_listener = start_queue_listener(log_queue, handler)
        self.broker = RabbitBroker(settings.rabbitmq_url, logger=root_logger)
        self.broker.include_router(router)
        self.app = FastStream(self.broker, logger=root_logger)
        self.container = get_notifier_container()
        setup_dishka(container=self.container, app=self.app)
        self.app.on_shutdown(self.shutdown_resources)

    async def shutdown_resources(self):
        get_notifier_guard().start_shutdown()
        await asyncio.sleep(_GRACEFUL_SHUTDOWN_DELAY_SECONDS)
        if self.broker:
            await self.broker.stop()
        if self.container:
            await self.container.close()
        if self.queue_listener:
            self.queue_listener.stop()

def init_app() -> FastStream:
    return FastStreamApp().app

router = RabbitRouter(handlers=v1_notification_consumers)

app = init_app()
```

Переводим приложение в режим завершения (`is_accepting=False`), даём воркерам время закончить текущие задачи и затем корректно останавливаем остальные ресурсы. В примере используется фреймворк `Faststream` на `rabbitmq`.

Можно сделать через хрупкий счетчик активных задач, но это лишено смысла - мы все равно упираемся в `graceful timeout` контейнера. Простое ожидание надежнее в этом случае.

По сути получается так: активный процесс после сигнала перестает обрабатывать новые задачи, дорабатывает текущие и корректно завершается.

Также уточню, что пример был приведен для однопоточного асинхронного воркера. Реализация может зависеть от архитектуры приложения и способа организации обработки задач. Например, если под капотом фреймворка используется `Process Pool`, важно учитывать, что механизм переключения состояния (через `is_accepting`) должен быть инициализирован в каждом дочернем процессе! Иначе консумеры в дочерних процессах будут игнорировать сигнал завершения и продолжать обработку новых задач. Возможно, что стоит подумать о распределенном ключе, который определяет активный деплой, но в нашем случае это усложнение.

В деплой скрипте логика такая: определяем активный инстанс (через регулярку/сопоставление/etc), сразу поднимаем `inactive` инстанс. В итоге на короткое время у нас работают оба инстанса и вместе обрабатывают задачи. Дальше отправляем `sigterm` активному инстансу - он переключается в режим неактивного (`is_accepting=False`), перестает обрабатывать новые задачи и спокойно дожидается завершения текущих.

Вот пример скрипта деплоя:

```
set -e

DEPLOY_PATH="$1"
COMPOSE="docker compose -f ${DEPLOY_PATH}/docker-compose.prod.yml"

if docker ps --format '{{.Names}}' | grep -q "^notifier-blue$"; then
    ACTIVE="blue"
    INACTIVE="green"
else
    ACTIVE="green"
    INACTIVE="blue"
fi

echo "[notifier] active=${ACTIVE}, deploying to=${INACTIVE}"

$COMPOSE up -d notifier-${INACTIVE}

$COMPOSE stop notifier-${ACTIVE}

echo "[notifier] done"
```

Объяснить с  SourceCraft

P.S. Для идемпотентных консьюмеров все еще проще — можно почти ничего не делать =) достаточно рейзить специальную ошибку с `requeue=True`, и условный RabbitMQ сам отправит сообщение обратно в очередь. Но проблема возникает с заполнением очереди, т.к. во время деплоя мы не будем обрабатывать сообщения. С неидемпотентными сообщениями такой подход уже проблемный — при повторной обработке мы получим неконсистентное состояние (например, упадем при проверке `id` сообщения). А неидемпотентных сообщений, как правило, большинство.

Для веб сервисов

В конфиге у нас всегда есть 2 инстанса приложения — `blue` и `green`. Также нужен реверс-прокси, например `nginx`.

Логика переключения реализована в деплой скрипте. Сначала определяем, какой инстанс сейчас активный. Это можно сделать через `if` логику: если активен `blue` - `Active = blue`, `Inactive = green`, в любом другом случае - наоборот `Active=green`, `Inactive = blue`. Определить это можно по регулярке/активному порту/etc. После этого запускаем `inactive` инстанс и проверяем его через `healthcheck`.

Дальше переключаем `nginx` на новый инстанс, что-то типа: `echo "server ${HOST}${INACTIVE_PORT};" > "$UPSTREAM_CONF"` и делаем мягкий бесшовный `reload(nginx -s reload)`. После этого устанавливаем время ожидания, равное максимальному времени выполнения `http` запроса в вашем сервисе (с запасом). Затем старому инстансу посылаем `sigterm` сигнал.

Вот пример скрипта деплоя:

```
set -e

IMAGE="$1"
DEPLOY_PATH="$2"
```

```

BLUE_PORT=${BLUE_PORT:-8002}
GREEN_PORT=${GREEN_PORT:-8003}

UPSTREAM_CONF="/etc/nginx/snippets/web-upstream.conf"
COMPOSE="docker compose -f ${DEPLOY_PATH}/docker-compose.prod.yml"

if grep -q "${BLUE_PORT}" "$UPSTREAM_CONF"; then
    ACTIVE="blue"
    INACTIVE="green"
    INACTIVE_PORT=$GREEN_PORT
else
    ACTIVE="green"
    INACTIVE="blue"
    INACTIVE_PORT=$BLUE_PORT
fi

echo "[blue-green] active=${ACTIVE}, deploying to=${INACTIVE} (port ${INACTIVE_PORT})"

$COMPOSE up -d web-${INACTIVE}

echo "[blue-green] waiting for health..."
for i in $(seq 1 30); do
    if curl -sf "http://127.0.0.1:${INACTIVE_PORT}/v1/health" > /dev/null 2>&1; then
        echo "[blue-green] healthy after ${i} attempts"
        break
    fi
    if [ "$i" -eq 30 ]; then
        echo "[blue-green] health check failed, rolling back"
        $COMPOSE stop web-${INACTIVE}
        exit 1
    fi
    sleep 2
done

echo "server 127.0.0.1:${INACTIVE_PORT};" > "$UPSTREAM_CONF"
nginx -t && nginx -s reload
echo "[blue-green] nginx switched to ${INACTIVE}"

sleep 5

$COMPOSE stop web-${ACTIVE}
echo "[blue-green] stopped web-${ACTIVE}, deploy complete"

```

Объяснить с 

Немного о миграциях

В blue green деплое важно, чтобы новая и старая версии приложения могли одновременно работать с одной схемой бд. Поэтому используем backward compatible подход: Например, нам нужно удалить атрибут или целую таблицу - сначала убираем её использование в коде и делаем деплой. После переключения трафика и завершения работы старых инстансов выполняем второй деплой с миграциями на удаление. Да, больше работы - но это того стоит.

Пример конфигурации Docker Compose

Вот фрагмент докер композ файла:

```
services:
  ...

  web-blue:
    image: ${CI_REGISTRY_IMAGE}/web:${IMAGE_TAG:-latest}
    container_name: web-blue
    restart: always
    env_file:
      - .env.prod
    ports:
      - "127.0.0.1:8002:8000"
    depends_on:
      migrations:
        condition: service_completed_successfully
      postgres:
        condition: service_healthy
      rabbitmq:
        condition: service_healthy

  web-green:
    image: ${CI_REGISTRY_IMAGE}/web:${IMAGE_TAG:-latest}
    container_name: web-green
    restart: always
    env_file:
      - .env.prod
    ports:
      - "127.0.0.1:8003:8000"
    depends_on:
      migrations:
        condition: service_completed_successfully
      postgres:
        condition: service_healthy
      rabbitmq:
        condition: service_healthy

  worker-blue:
    image: ${CI_REGISTRY_IMAGE}/worker:${IMAGE_TAG:-latest}
    container_name: worker-blue
    restart: always
    stop_grace_period: 200s
    env_file:
      - .env.prod
    depends_on:
      migrations:
        condition: service_completed_successfully
      postgres:
        condition: service_healthy
      rabbitmq:
        condition: service_healthy
      redis:
        condition: service_healthy
```

```
worker-green:
  image: ${CI_REGISTRY_IMAGE}/worker:${IMAGE_TAG:-latest}
  container_name: worker-green
  restart: always
  stop_grace_period: 200s
  env_file:
    - .env.prod
  depends_on:
    migrations:
      condition: service_completed_successfully
    postgres:
      condition: service_healthy
    rabbitmq:
      condition: service_healthy
    redis:
      condition: service_healthy

notifier-blue:
  image: ${CI_REGISTRY_IMAGE}/notifier:${IMAGE_TAG:-latest}
  container_name: notifier-blue
  restart: always
  stop_grace_period: 15s
  env_file:
    - .env.prod
  depends_on:
    migrations:
      condition: service_completed_successfully
    postgres:
      condition: service_healthy
    rabbitmq:
      condition: service_healthy
    redis:
      condition: service_healthy

notifier-green:
  image: ${CI_REGISTRY_IMAGE}/notifier:${IMAGE_TAG:-latest}
  container_name: notifier-green
  restart: always
  stop_grace_period: 15s
  env_file:
    - .env.prod
  depends_on:
    migrations:
      condition: service_completed_successfully
    postgres:
      condition: service_healthy
    rabbitmq:
      condition: service_healthy
    redis:
      condition: service_healthy
```

...

Заключение

В итоге получился вполне рабочий blue-green деплой с нулевым даунтаймом, и всё это на обычном докер композ.

Если есть идеи, как сделать проще и надежнее, или замечания по подводным камням, которые я мог не учесть — делитесь, буду рад почитать!

Теги: [blue-green deployment](#), [zero downtime](#), [docker-compose](#), [cicd](#), [rabbitmq](#), [kafka](#), [faststream](#), [celery](#)

Хабы: [Python](#), [Django](#), [Веб-разработка](#), [Говнокод](#), [Микросервисы](#)



Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Подписаться

Оставляя почту, я принимаю [Политику конфиденциальности](#) и даю согласие на получение рассылок



4K+

1

Охват за 30 дней

Карма

Лев @merra123

Программист

Подписаться



Поток Бэкенд доступен 24/7 благодаря поддержке друзей Хабра

Хабр Курсы для бэкендеров

РЕКЛАМА

Практикум, Хекслет, SkyPro, авторские курсы — собрали всех и попросили скидки. Осталось выбрать!

Перейти

[Перейти в поток Бэкенд](#)

Комментарии 4

Публикации

ЛУЧШИЕ ЗА СУТКИ

ПОХОЖИЕ



the_bat

23 часа назад

Ремонт блока питания с Power Delivery. 470 граммов электроники

Простой

4 мин

16K

+96

30

27



interpres

18 часов назад

Электроинструмент становится хуже, и это делается намеренно

Простой 11 мин 26K

Обзор

Перевод

+71

45

82



prohronus

19 часов назад

Как ИИ-агенты стали новым оружием скамеров на Хабр Карьере

5 мин 9.3K

Кейс

+51

9

9



alizar

22 часа назад

Готовимся к отключению. Эффективные форматы для упаковки и раздачи HTML-страниц

Простой 7 мин 12K

Обзор

+37

59

18



TrexSelectel

19 часов назад

Linux 7.0 и что изменилось в ядре после очередного цикла разработки

5 мин 8.7K

Обзор

+27

10

0



AlexSerbul

23 часа назад

Программирование с AI-ассистентом — похороните меня под плинтусом

Средний 14 мин 9.9K

Мнение

+26

33

11



mariklolik

18 часов назад

Как переложить нагрузку по code review с разработчиков на LLM

Средний 7 мин 7.4K

Кейс

+23

23

6

nick_dyuba
22 часа назад

Легенды 90-х — кто придумал и производил жвачки Turbo, Love is... и TipiTip

5 мин 7.1K

Recovery Mode

+22

8

5

Mimizavr
23 часа назад

«Великое очищение» в работе с контентом: что осталось от роли редактора

11 мин 7K

+17

9

7

infgotoinf
10 часов назад

Perl — зря забытый язык программирования?

11 мин 8.5K

Из песочницы

+16

13

8

Big Data в фарме: как мы управляем сотнями сотрудников по всей стране

Турбо

Показать еще

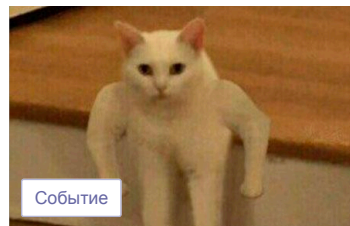
МИНУТОЧКУ ВНИМАНИЯ



Собери облачный пазл и выиграй призы



Удалёнка в фарме: большие данные и никакой магии



Качаем к лету хард- и софт-скиллы на айтишных ивентах

КУРСЫ

Web-разработчик с нуля: профессия с выбором специализации

По мере набора группы

 Python-разработчик

По мере набора группы

 Профессия Инженер по тестированию + ИИ

По мере набора группы

 Профессия: Python-разработчик + ИИ

По мере набора группы

 Python-разработчик

По мере набора группы

[Больше курсов на Хабр Карьере](#)

ЧИТАЮТ СЕЙЧАС

Электроинструмент становится хуже, и это делается намеренно

 26K  82 

Тим Кук покидает пост Apple. Новый глава — «отец» Apple Silicon Джон Тернус

 13K  14 

Представлен проект KillerPDF — редактор PDF с открытым исходным кодом для Windows 10/11

 6.9K  15 

Пользователь купил Atomic Heart на VK Play, но не смог запустить игру из-за сетевых проблем

 5.4K  11 

+185% за 13 часов: как Kimi K2.6 переписала 8-летний движок

 14K  10 

Big Data в фарме: как мы управляем сотнями сотрудников по всей стране

Турбо

ИСТОРИИ



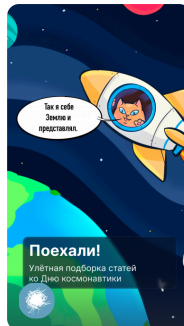
Годнота из блогов компаний



Только 10% решат эту головоломку



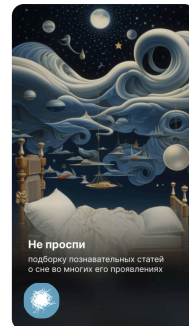
Хочу 450К



Вперёд к звёздам



Там на неведомых дорожках...



Самое время поспать

БЛИЖАЙШИЕ СОБЫТИЯ



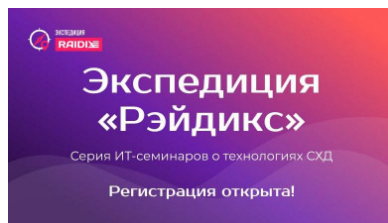
2 марта – 10 августа

**PROBA Awards —
международная
коммуникационная
премия, учрежденная в
2000 году**

Санкт-Петербург

Маркетинг

[Больше событий в календаре](#)



3 марта – 9 июня

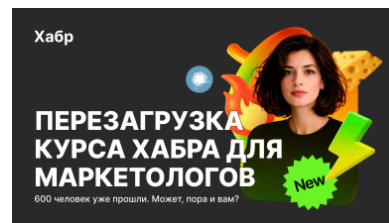
Экспедиция «Рэйдикс»

Екатеринбург • Новосибирск •
Краснодар • Минск • Калининград •
Ростов-на-Дону

Разработка

Другое

[Больше событий в календаре](#)



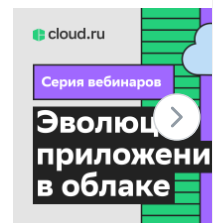
9 марта – 1 июня

**Всё, что нужно, чтобы
стартовать продвижение
бренда на Хабре**

Онлайн

Маркетинг

[Больше событий в календаре](#)



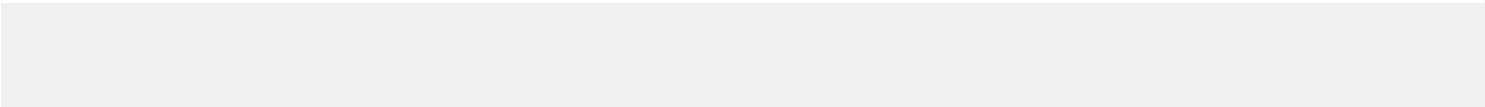
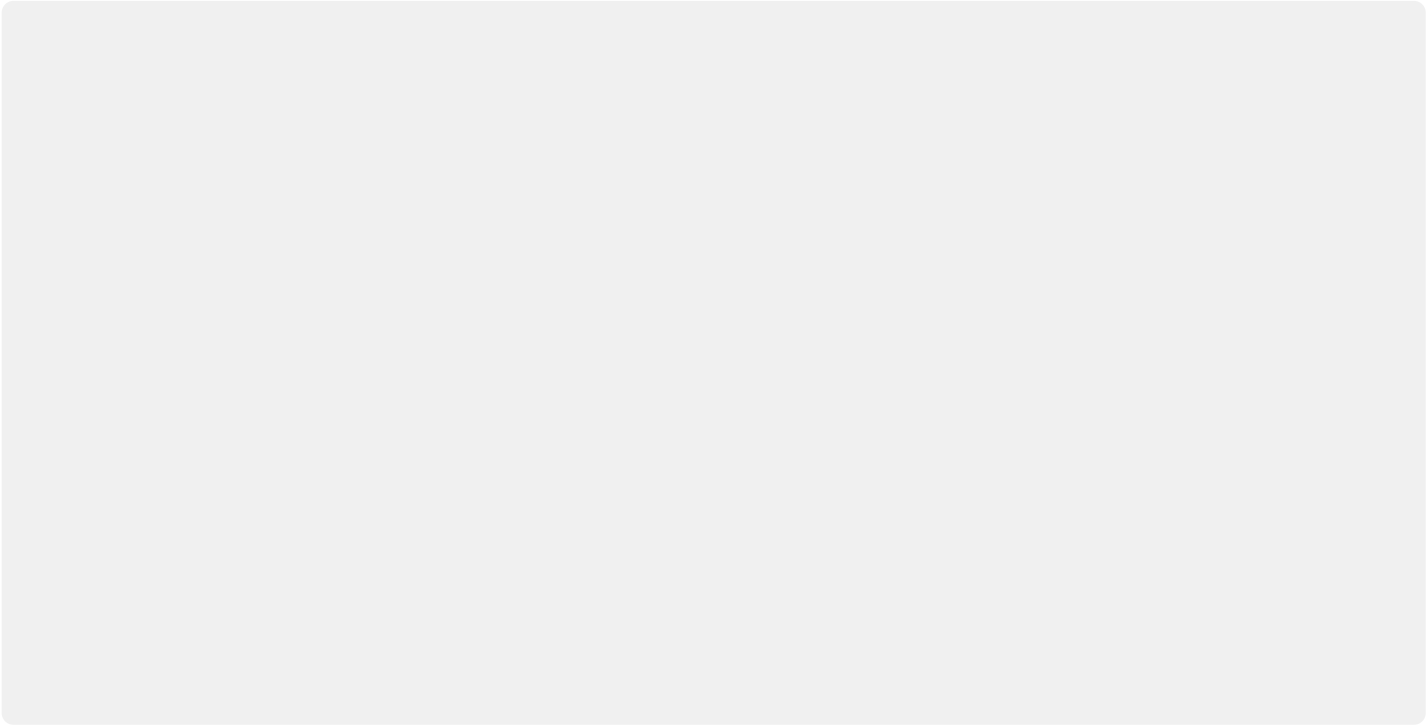
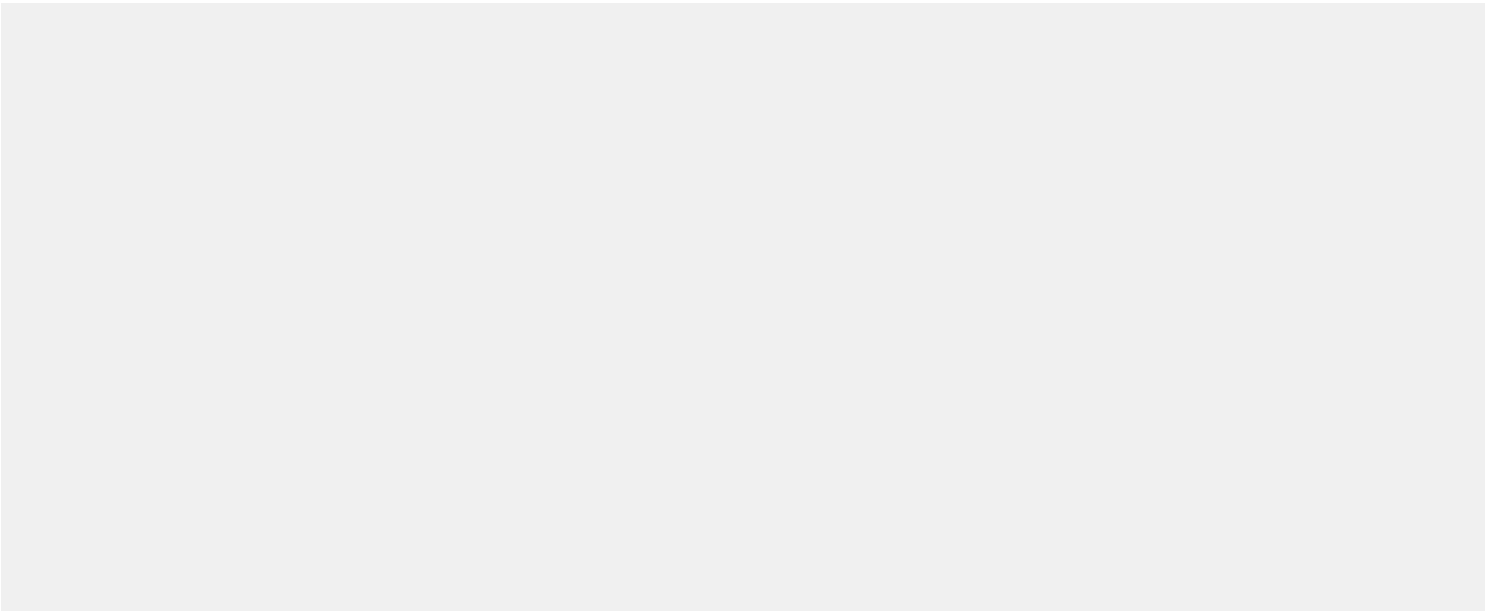
19 марта – 28 апреля

**Серия вебинаров
«Эволюция п
в облаке»**

Онлайн

Разработка

[Больше событий в календаре](#)



Ваш аккаунт	Разделы	Информация	Услуги
Войти	Статьи	Устройство сайта	Корпоративный блог
Регистрация	Новости	Для авторов	Медийная реклама
	Хабы	Для компаний	Нативные проекты
	Компании	Документы	Образовательные программы
	Авторы	Соглашение	Стартапам

[Песочница](#)

[Конфиденциальность](#)



[Настройка языка](#)

[Техническая поддержка](#)

© 2006–2026, Habr